



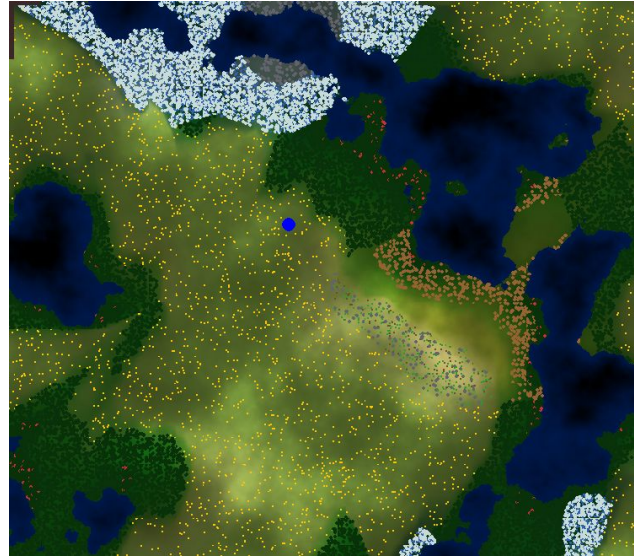
Procedural Generation of a Coherent Infinite 2D Landscape

José Martins, Francisco Cardoso, Cedric Hartz



Procedural Generation of a Coherent Infinite 2D Landscape

- Create an Infinite procedurally generated 2D World using Pygame.
- Reproduce Physics instead of using stochastic biome placing
- Biomes appear naturally where they should depending on many factors.

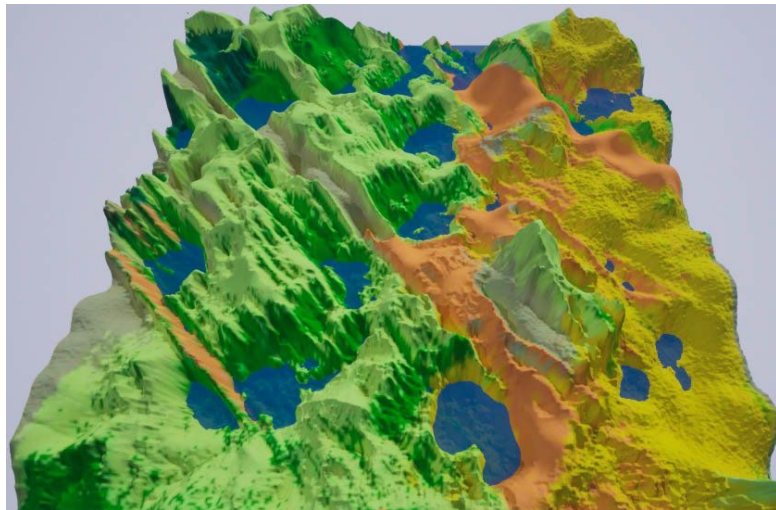




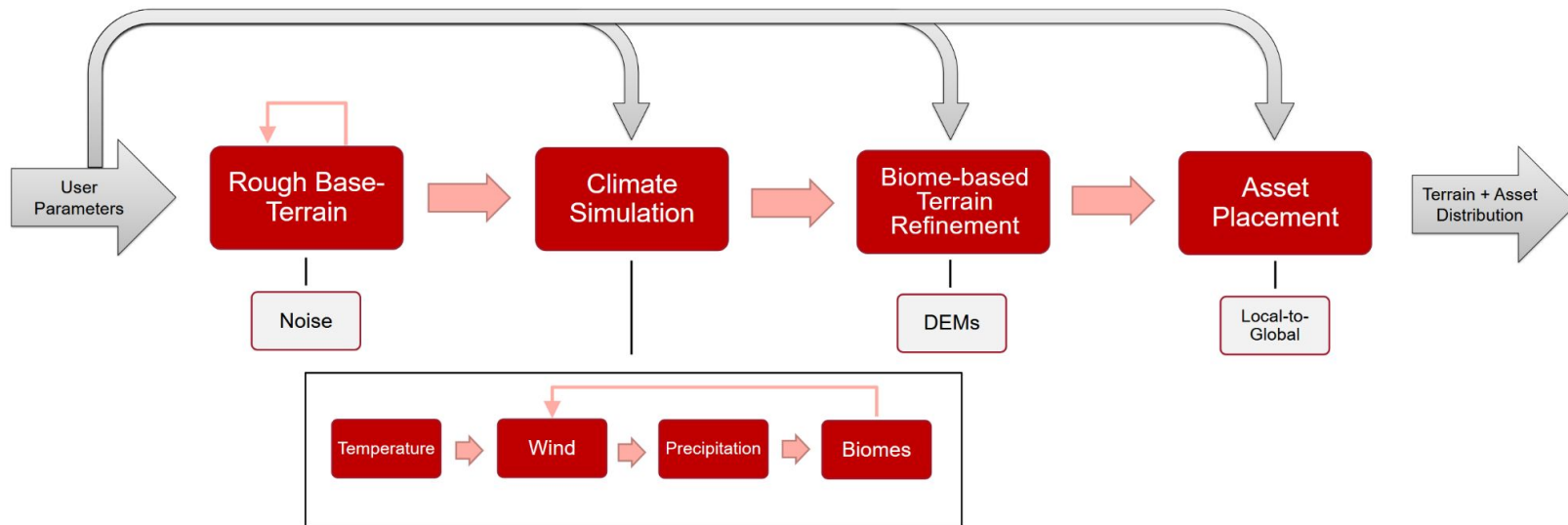
AutoBiomes

AutoBiomes (Chen et al., 2020)

- Unreal Engine 4 framework.
- New approach that deviates from the common Noise→Heightmap→World.
- Our project bases itself on this pipeline, but in 2D.



The Pipeline

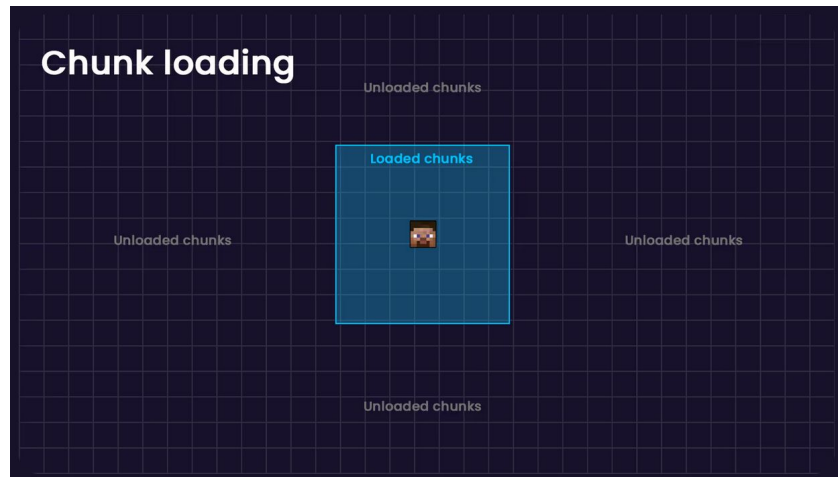




The Challenge of Infinity

The Constraint: Local vs. Global Generation

- Physics Constraints.
- Cannot generate the full-map, there is not infinite memory.
- Cannot simulate wind blowing through a mountain that hasn't been generated yet.
- Most of the “non-noise content” must be local.

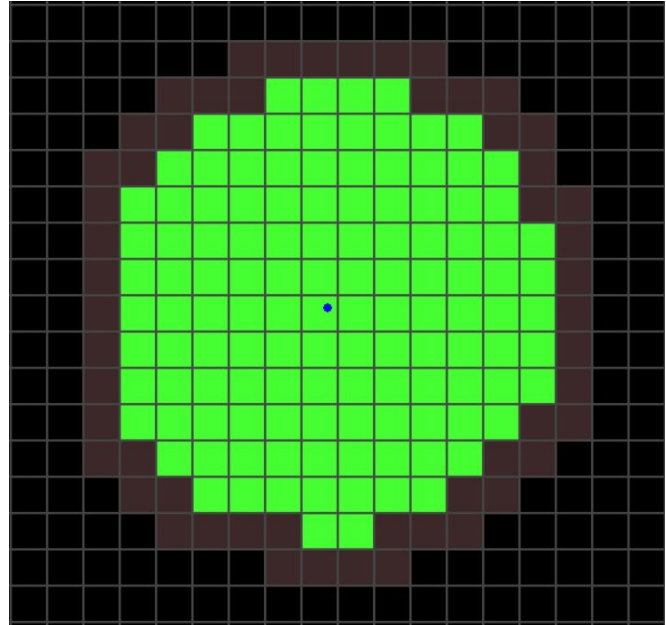




Our Approach

Chunk System

- Infinite world → Chunks.
- When generating, a chunk needs to know its neighbors.
- Original idea was to smooth the loading of chunks using the neighbors system.
- In the end, there is a radius for loading and unloading chunks.



Noise Engine

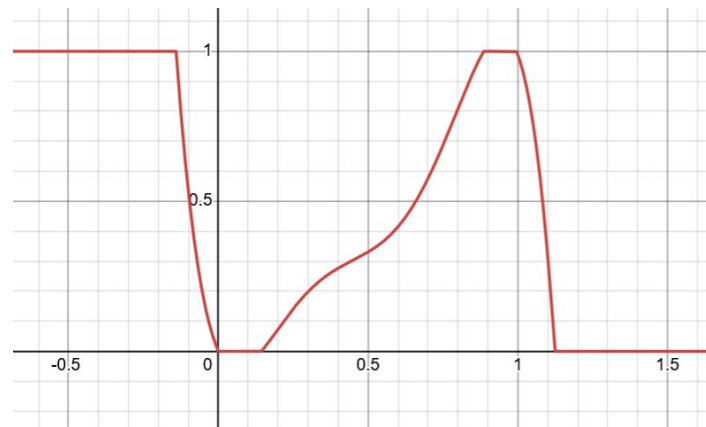
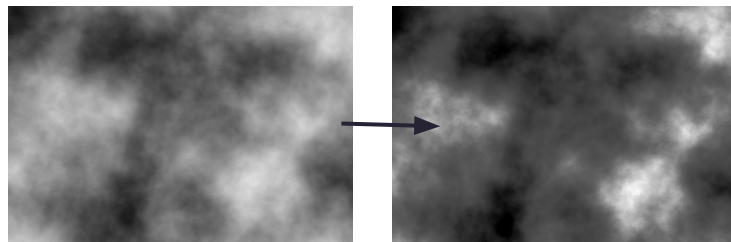
- Similar to AutoBiomes.
- Uses pyfastnoiselite python library.
- Uses Simplex Noise and fBm (Fractional Brownian motion).
- Generates HeightMaps.
- Altitude, Temperature and Wind are generated with it.





Altitude reshaping

- Apply function $[0,1] \rightarrow [0,1]$ per pixel
- More spiky mountain peaks
- More plateaus
- Fast using numpy



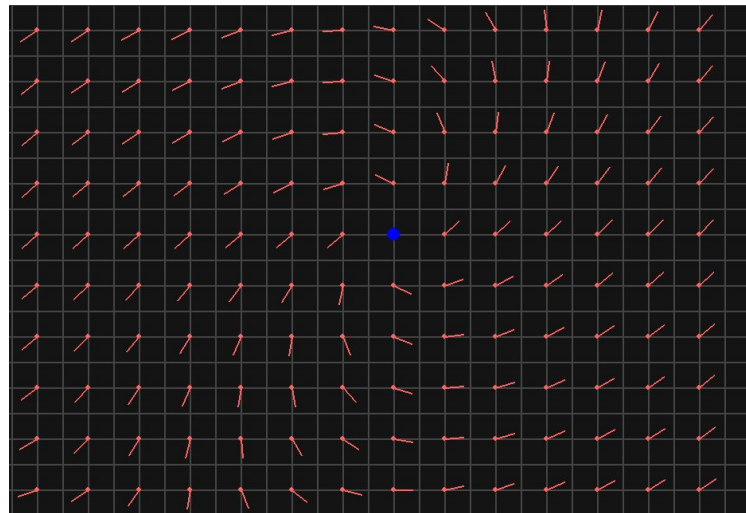
Generating a Global Wind System

Vector Field: Unlike altitude, wind is a vector.

The Math: Created using two separate Perlin Noise maps:

- Map A $\rightarrow$ X-Component (East/West)
- Map B $\rightarrow$ Y-Component (North/South)

Result: Smooth, continuous wind currents that span the entire infinite world.

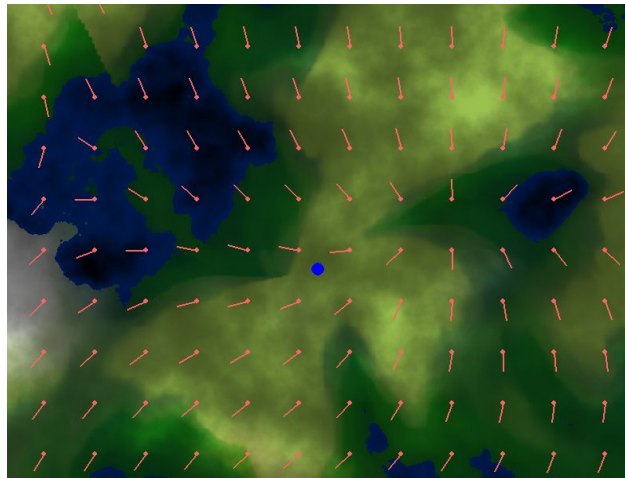


Simulating Moisture via "Reverse Tracing"

Reverse Tracing: For every pixel, we trace a "virtual raindrop" backwards against the wind.

"Where did this air come from?"

- If it came from the Ocean → It brings Rain.
- If it traveled far over Land → It is Dry.



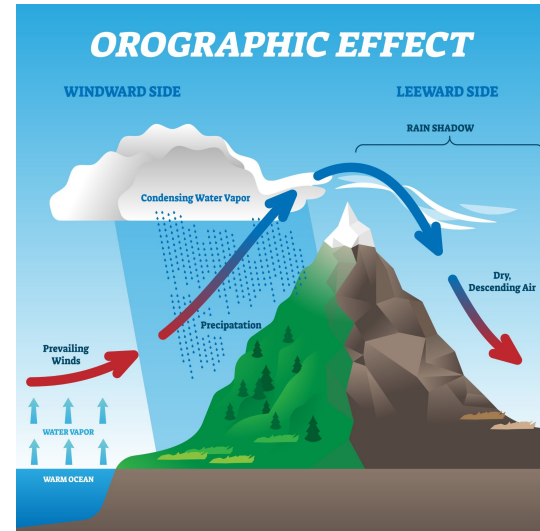
Precipitation

Moisture Pickup: Air gains moisture over oceans.

Continental Decay: Air slowly loses moisture as it travels inland (creating Coast → Forest → Grassland transitions).

Rain Shadow Effect: If the trace hits a Mountain (> 0.6), moisture is mostly blocked.

Outcome: Realistic deserts form naturally behind mountain ranges.



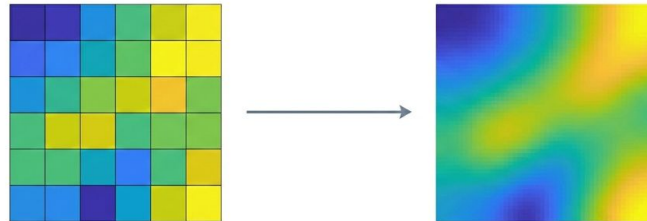
Optimization: Bilinear Upscaling

Problem: Tracing wind 80 steps for every single pixel is slow (1024 traces per chunk).

Solution:

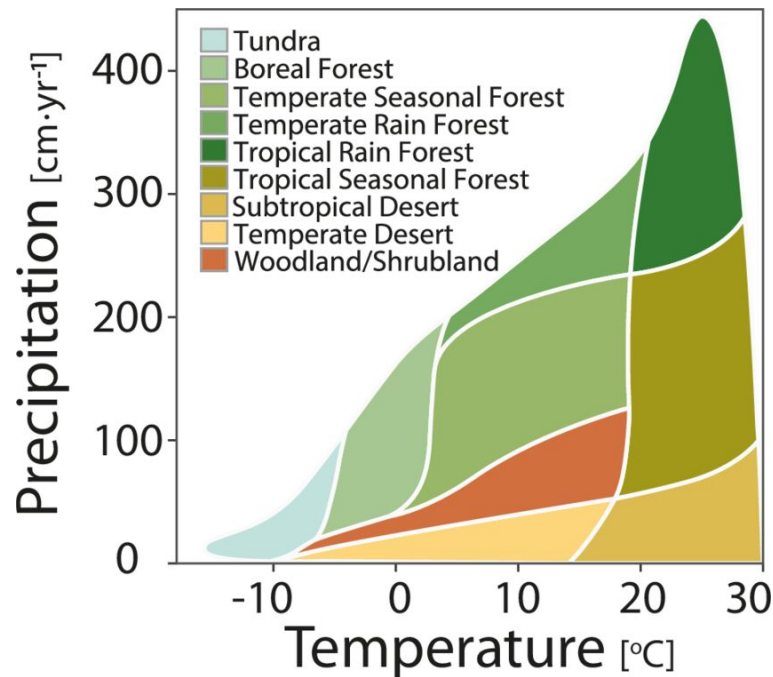
- Calculate moisture for a low-res grid (8×8).
- Use Bilinear Interpolation to upscale to the full chunk (32×32).

Result: 16x Performance Gain with no visual loss.



Biome Placement

- Calculated using a Whittaker Diagram
- Rule based
- N biomes
- Temperature and precipitation are both environmental maps
- At the edges, biomes are blended together
- biome-map contains N-dimensional vectors (10 biomes -> 10 dimensional vector)
- e.g. (0, 0, 0.06, 0.21, 0.54, 0, 0.19, 0, 0, 0)
- probability vector sums up to 1
- fast using pre-rendered look up table



Object Placement: Deterministic Randomness

The Challenge:

- **Order Dependency:** Standard random generators depend on the order call.
- **Travel Problem:** A player might visit Chunk A → Chunk B, or Chunk B → Chunk A.
- **The Consequence:** Trees would disappear or move every time you revisited an area.

The Solution:

- No global random state.
- Unique seed derived directly from the chunk's coordinates.
- The random numbers are "locked" to the location.

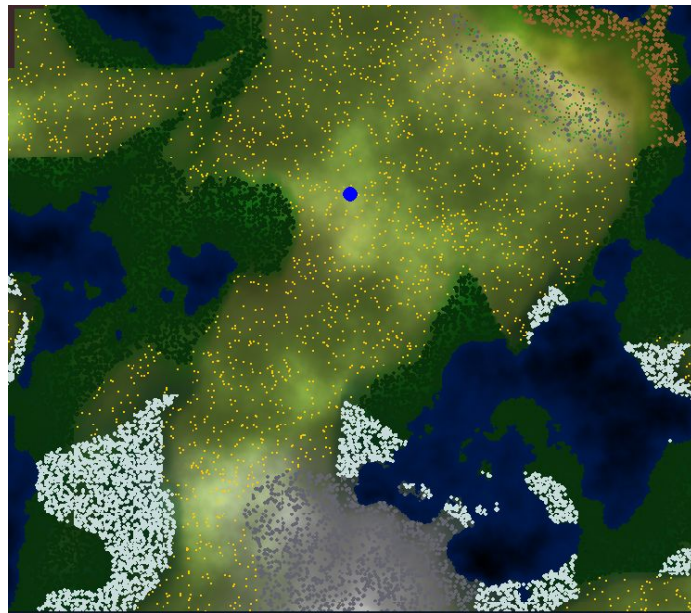
Biome-Specific Object Rules

Forests: High density of Snow, Tropical or normal Trees. (Depending on the biome) Mushrooms and Berry bushes too.

Desert: Low density of Cacti, Dead Bushes and rocks.

Tundra: No vegetation, but geological features (Rocks).

Grasslands: Medium density of yellow flowers





Demo



Future Work

The Challenge of Infinite Rivers

The Problem: Global Flow in a Local Simulation

- Gravity requires knowledge
- The "Local Minimum" Issue
- Connectivity

Proposed Solution: Inverted Generation ("Carving")

- Don't simulate the water; simulate the valley.
- River Noise Map:

The Logic:

- Generate the "River Noise".
- If the value is high, we assume a river exists there.
- Carve the Altitude: We mathematically lower the terrain altitude at that spot to create a riverbed.
- Force Biome: We explicitly set that pixel's biome to "River".

A low-poly landscape illustration. The scene features rolling green hills, a winding blue river, and several brown, rocky mountains. In the distance, a small village with red-roofed buildings is visible on a hill. The sky is blue with a few white clouds. The overall style is geometric and stylized.

Thank you.
Any Questions?